ML Challenge

Sensor Context Encoder Challenge

Objective

Build a small model that converts a window of inertial-sensor data into **one continuous context embedding** that can be read by a frozen language model.

Compare it against a normal sensor classifier and decide whether the language-model path is useful enough to continue developing.

This is not based on the assumption that a language model will classify activities better. A direct classifier may be faster, smaller, and more accurate. The purpose is to test whether a sensor can be connected to a shared language-model interface without converting its values into text and being able to understand raw sensor values via tokens as the interface.

Problem statement

A future on-device model may need to consume several continuous signals such as: proximity, light, sound direction, motion, emotion, robot state, along with language.

One possible design is to encode each signal as a small number of vectors in the language model's embedding space. This challenge tests that idea with one sensor modality and one simple task.

A **continuous context embedding** is a learned vector with the same dimension as a language-model token embedding. It summarizes the complete sensor window, but it is not a word and does not correspond to an entry in the model's vocabulary.

sensor window → sensor encoder → projector → context embedding → frozen LLM → activity prediction

Dataset and task

Use the UCI Human Activity Recognition Using Smartphones dataset.

Use only the nine windowed inertial signals in the Inertial Signals folders:

- total_acc_x, total_acc_y, total_acc_z
- body_acc_x, body_acc_y, body_acc_z
- body_gyro_x, body_gyro_y, body_gyro_z

Each input has a shape 128 x 9. Do not use the supplied 561-dimensional engineered features.

Predict one of the six activities:

walking, walking_upstairs, walking_downstairs, sitting, standing, laying

Use the official subject-wise train/test split. Create validation data using subjects from the training split only. Do not tune the model using the test set.

Language model

Use: **HuggingFaceTB/SmolLM2-360M-Instruct**

Keep all language-model parameters frozen, including its token-embedding table.

Use this prompt:

Classify the activity as walking, walking upstairs, walking downstairs, sitting, standing, or laying.

Sensor context: <SENSOR>

Activity:

Replace <SENSOR> with the projected context embedding. It must be inserted directly into the language model's input-embedding sequence; the sensor values must not be converted into text.

Read the final hidden state after Activity: and pass it through a trainable linear layer with six outputs. Free-form text generation is not required.

Train only:

- Sensor encoder
- Projector
- Classification head

The sensor encoder, projector, and head together must contain no more than 10 million trainable parameters. Provide justification if you need/use more.

What to build

1. Direct sensor classifier

Train a normal classifier using the same sensor-encoder architecture:

sensor window → sensor encoder → linear classification head

This is the engineering baseline. If activity recognition were the only goal, this may be the better solution.

2. Context-embedding model

Train the sensor encoder and projector to produce one continuous embedding for the frozen language model:

sensor window → sensor encoder → projector → frozen LLM → linear head

The direct and context models are trained separately, but must use the same train, validation, and test data.

3. Sensor-dependence check

Evaluate the trained context model again after shuffling its projected embeddings between test examples.

Shuffle complete embeddings, not individual values, and do not retrain the model. This breaks the connection between a sensor window and its activity while keeping the embeddings themselves realistic.

If performance does not drop meaningfully, the model may not be using the matching sensor input.

Technical freedom

You may choose the sensor-encoder architecture, projector design, optimizer, scheduler, regularization, augmentation, batch size, validation subjects, and repository structure.

Training may use a local or cloud GPU. No live phone integration, language generation, or language-model fine-tuning is required.

Timebox

Spend no more than **two to three days**. A small, correct, well-explained prototype is preferred over additional incomplete experiments.

Use of AI tools

AI-assisted development tools are allowed. You remain responsible for the code, experiments, results, and technical decisions, and should be able to explain and modify the work.

Submission

Submit the following:

1. **Source code and dependencies** for training and evaluating the direct classifier, context-embedding model, and shuffled-embedding check.
2. **README** with setup instructions and the commands needed to reproduce the results.
3. **Results table** containing macro-F1 for all three required conditions.
4. **Technical note**, no more than two pages, containing:
    - The sensor encoder and projector design
    - Training setup and trainable parameter count
    - The results and their interpretation
    - Known limitations
    - Recommendation on whether the context-embedding approach should be developed further.

One reproducible run per condition is enough. Include the seed used.

Results table

Condition	**Macro-F1**
Direct sensor classifier	
Context-embedding model	
Context model with shuffled embeddings	

Evaluation criteria

- Correct and reproducible implementation.
- Fair comparison between the direct and context models.
- Complete frozen-backbone and continuous-embedding setup.
- Valid sensor-dependence check.
- Appropriate scope and technical judgment.
- Clear interpretation and recommendation.